Decision Orchestration: Learning Annotation Procedures from a Handful of Gold Documents

Anonymous ACL submission

Abstract

A new annotation scheme may have a guideline but little gold data. Whole-graph prompting can violate conventions and produce malformed output, while supervised parsing often requires substantially more labeled examples. We observe that a guideline is mostly a procedure: most steps are deterministic, and only a few call for judgment. We propose *decision orchestration*, which manually encodes a guideline as an executable procedure separating rule nodes from model decision nodes. Code runs the steps, assembles and validates the output, and a model is consulted only at decision points, for a schema-constrained fragment or for probabilities over the options of a closed question. We then learn the procedure itself, not model weights, from a handful of gold documents. Decision thresholds are optimized against the task metric, uncertain decisions are routed to a stronger model, and reflection uses aligned traces to localize discrepancies. In closed mode, proposed edits are retained through measured trials; fragment-mode edits are unmeasured. On Chinese Uniform Meaning Representation parsing, the 9B fragment procedure scores 17.6 F against 0 for a minimal prompt and 4.9 for a detailed prompt, two gold documents raise a frontier model from 45.3 to 52.6 F, recalibration on four documents adapts a new decision model (+2.9 F), and routing recovers 75% of a stronger model's gain at approximately 26% of its reported API cost on a 20-sentence pilot. Evidence is limited to Chinese UMR; common-test-set baselines, repeated runs and the proposed second benchmark remain pending.

1 Introduction

New annotation schemes often start with a prose guideline and little gold data. Producing the first annotations is costly (Settles, 2009), particularly for Uniform Meaning Representation (UMR; Van Gysel et al., 2021), which requires sentence graphs and document-level temporal, modal and coreference relations. Its release has only a few hundred gold graphs per major language (Bonn et al., 2025). LLMs can assist familiar labeling tasks (Gilardi et al., 2023; Tan et al., 2024), but whole-graph prompting can violate semantic-representation conventions and produce malformed output (Ettinger et al., 2023; Sun et al., 2024). Our minimal Sonnet prompt reaches 29–33 sentence F; a 9B model yields no scorable graphs, although a detailed prompt reaches 4.9 F.

We propose **decision orchestration** (Figure 1): manually encode a guideline as an executable procedure, separating deterministic dispatch and assembly from local model judgments. For UMR, code controls frame lookup, typed dispatch, coverage and graph assembly; models decide span types, senses, roles and relations. Each decision node returns either a schema-constrained fragment or distributions over closed options enumerated by code. Typed answers affect tree shape and call counts, but cannot invoke arbitrary skills. The same interface supports fragment-producing LLMs and closed-question services (TypeSafe, 2026; OpenAI, 2026; Microsoft, 2026).

The procedure is then adapted under gold supervision without updating model weights. **Threshold tuning** optimizes how rules interpret answer probabilities against the task metric; it does not calibrate those probabilities. **Routing** replaces uncertain answers with a stronger model's answers. **Reflection** uses aligned traces to propose local edits. Output provenance helps inspection but is not causal attribution. Closed-mode edits undergo one-at-a-time measurements on training, regression and development documents; fragment-mode edits are unmeasured. Every revision is a readable artifact.

The current evidence is from Chinese UMR. Two fragment-mode reflection documents raise Sonnet from 45.3 to 52.6 F on T1. Closed-mode runs inherit additional manual seed tuning and development exposure (§5); threshold adaptation on

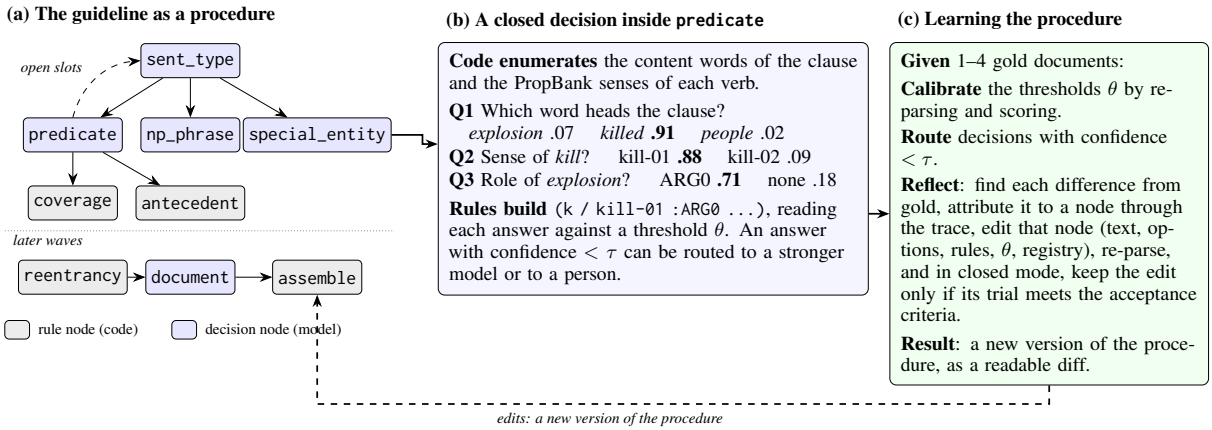


Figure 1: Decision orchestration. (a) The guideline is compiled into skills dispatched by a registry. Rule nodes are code; decision nodes consult a model. Shown is part of the UMR procedure (all nodes: Table 8, appendix). (b) In closed mode, a decision node asks questions whose options code has enumerated, and rules read the answer probabilities. (c) With a handful of gold documents, the procedure itself is learned. Example sentence, in English for readability: *Police said the explosion killed two people.*

four documents adds 2.9 F on H. In a 20-sentence routing pilot, uncertain-question escalation recovers 75% of the stronger answerer's gain at approximately 26% of its reported API cost. Training gains exceed test gains, and common-test-set baselines, repeated runs and a second benchmark remain pending.

Our contributions are the manually initialized procedure and interchangeable decision interfaces, adaptation through threshold tuning, routing and supervised edits, and an exploratory study of their behavior in Chinese UMR.

Scope and shared infrastructure. The companion study *Can LLMs Build Better Annotators? Self-Improving Harnesses for Ontology-Guided Annotation* examines instruction revision in a three-call whole-graph harness. It shares our UMR data, alignment and judge and reuses procedural runs for comparison. Here we study dispatch, assembly and decision interfaces; shared runs are not independent replication.

2 Related Work

LLMs as annotators. LLMs can assist crowd labeling (Wang et al., 2021; Gilardi et al., 2023; He et al., 2024; Ziems et al., 2024; Tan et al., 2024). Human–LLM collaboration allocates uncertain items or verifies labels (Li et al., 2023; Wang et al., 2024); we route decisions within structured items. GoLLIE (Sainz et al., 2024) learns to follow code-form guidelines, Code4Struct (Wang et al., 2023) represents event structures as code, and guideline learning (Pang et al., 2023) induces text from errors. Here the harness executes a manually encoded procedure whose rules and thresholds can also change.

LM programs and structured output. Prompt chains and decomposed prompting compose model calls (Wu et al., 2022; Khot et al., 2023); programs can call models or declare their constraints (Gao et al., 2023; Gupta and Kembhavi, 2023; Beurer-Kellner et al., 2023; Khattab et al., 2024). Grammar constraints control output form (Willard and Louf, 2023; Geng et al., 2023), potentially affecting reasoning (Tam et al., 2024). Unlike model-directed agent control (Yao et al., 2023), our registry dispatches typed emits, keeps rule nodes explicit and allows closed distributions as an alternative to fragments.

Optimizing prompts and programs. Prompts can be searched or optimized from feedback (Zhou et al., 2023; Pryzant et al., 2023; Yuksekgonul et al., 2025; Yang et al., 2024; Opsahl-Ong et al., 2024). GEPA reflects on traces (Agrawal et al., 2026); ACE grows a playbook (Zhang et al., 2026). Workflows and scaffolds can also be searched (Zhang et al., 2025; Hu et al., 2025; Zelikman et al., 2024). Self-feedback alone is unreliable (Huang et al., 2024). Our supervised reflection uses structured gold alignment to propose node-local edits to options, rules and text, with measured selection in closed mode. This resembles transformation-based learning and data programming in producing readable artifacts (Brill, 1995; Ratner et al., 2017), with-

out claiming causal attribution from traces alone.

Threshold adaptation and cascades. Probability calibration and selective prediction study confidence and abstention (Guo et al., 2017; Jiang et al., 2021; Zhao et al., 2021; Kamath et al., 2020); cascades escalate to costlier models (Chen et al., 2024). We instead tune program thresholds against graph scores and escalate individual closed questions. Teacher agreement is reported separately from gold correctness.

Meaning representation parsing. Supervised AMR parsers use large labeled corpora (Bevilacqua et al., 2021; Zhou et al., 2021; Bai et al., 2022; Knight et al., 2020); UMR has less gold (Van Gysel et al., 2021; Bonn et al., 2023, 2024, 2025). Pipeline conversion (Chun and Xue, 2024) and SETUP (Markle et al., 2025) provide relevant parsing alternatives. Our matched comparisons with these and stronger prompting baselines remain pending.

3 Decision Orchestration

Setting. An annotation scheme defines a structured annotation y for a document x; in UMR, one rooted graph per sentence plus document-level temporal, modal and coreference relations. In the cold start we have a prose guideline, the external resources it refers to (here PropBank frames; Palmer et al., 2005), an automatic scorer, and a handful of gold documents. We train no model weights. We build, and later learn, a procedure Π that maps x to y.

Skills, registry and execution. Π consists of **skills**, a **registry** and numeric parameters θ. A skill has an input schema, an output schema (its *emit*) and an implementation: code (a *rule node*) or a model call (a *decision node*). The registry declares *waves* that run in order; for UMR these are sentence, reentrancy and document. Within a wave, a *dispatcher* maps fields of a skill's emit to the next skill. An *episode* starts at the wave's start skill (e.g., with a whole sentence) and grows a tree of nodes until every branch ends in a leaf. Control flow is thus a fixed function of typed decisions. The model cannot issue an arbitrary scheduling command or choose an undeclared skill. The dispatcher selects subsequent nodes from typed emits, so model answers and open slots can change tree shape and call counts. The output handler owns final formatting.

Table 8 in Appendix D lists the UMR nodes. `sent_type` classifies a span and routes it to `discourse`, `predicate`, `np_phrase` or `special_entity`, or stops it as a leaf. Graph-producing skills emit a fragment whose unexpanded parts are typed placeholders (*open slots*), each dispatched back to `sent_type`. Out-of-span participants go to the rule node `antecedent`, and the rule node `coverage` re-dispatches every token no child covers. The reentrancy wave merges coreferring nodes by rule. The document wave calls `document` once per sentence with its graph and the five preceding ones. An output handler assembles and validates the PENMAN graphs.

Two kinds of decision nodes. A decision node has two implementations that produce the *same* emit, so they can be swapped node by node. In a **fragment decision**, an LLM receives the skill's instructions (the relevant part of the guideline, rewritten for a model), the span and the numbered sentence. It may call `get_example` for worked examples and `propbank_lookup` for frames, and it answers through the emit schema, for example with a type from a closed list or a graph fragment with placeholders. It never writes a whole graph. In a **closed decision**, the skill's code enumerates candidates from the input (the words of the span, the senses and roles of a verb, the allowed relations, the preceding mentions) and asks *closed questions*: yes/no, a choice among options that each carry a one-line description from the guideline, or a graded score. A decision model returns a distribution over the options and no text. The skill is a coroutine: it yields a round of questions, receives the distributions, may ask a follow-up round, and returns an emit built by rules. `predicate`, for instance, asks three rounds (Figure 1b): the head and whether it is an abstract predicate; its sense, aspect, modal strength and polarity; and the role of every content word (one of the sense's roles, a general role, "part of the predicate" or "none"). Rules read the distributions against thresholds θ. For example, a span with a comma goes to `discourse` when $p(\text{discourse}) \geq 0.2$, and a coreference link is kept when $p \geq 0.4$. Closed mode needs no model that can write structured text, and every decision comes with a probability.

Traces and attribution. Every node records its input, its decision (in closed mode, the questions and full distributions) and its emit. After scoring, the scorer's alignment of predicted to gold variables

lets us record the producing node for an aligned predicted unit (a concept, role, attribute or relation). Missing gold-only units have no predicted producer. This is output provenance, not a verified causal attribution: upstream routing, candidate omissions and alignment errors can change downstream losses. A fine-grained judge scores 25 aspects separately, such as senses, aspect, core arguments, name tokens, reentrancy, and temporal and coreference relations (Appendix A). Summing the lost credit per skill shows where the procedure fails, and the same records are what reflection reads.

4 Learning the Procedure from a Handful of Documents

All learning here changes Π (text, code, thresholds, registry) and never a model. We measure a procedure on gold documents D by

$$J(\Pi; D) = \frac{1}{|D|} \sum_{d \in D} \tfrac{1}{2}\big(F_d + C_d\big), \qquad (1)$$

where F_d is the sentence-graph F-score and C_d the comprehensive score that also covers the document level (Sun and Xue, 2024). Without C_d, changes to the document-level skill could not be measured. The few documents play different roles. Errors are diagnosed on *training* documents. *Regression* documents (earlier training documents) must not get worse. *Development* documents are measured but never supplied for error diagnosis; repeated acceptance decisions nevertheless use their scores, so they are model-selection data. The test set is excluded from edit generation and acceptance, although authors inspect its frozen-version trajectory retrospectively.

4.1 Task-level threshold tuning

Closed-mode rules read probabilities against thresholds that were set for one decision model. Another model spreads its probability mass differently, so a threshold rule fires too often or too rarely and the tree changes shape. What matters is the decision a rule takes, not each probability (cf. Zhao et al., 2021). We therefore fit θ to J directly. In coordinate descent, we sweep one threshold at a time over a few values, re-parse the calibration documents, and keep a value if it raises J by more than 0.05. Ten thresholds with three or four values each take about 40 parses of four documents, which is minutes with a decision model.

4.2 Routing uncertain decisions

A closed answer has a confidence $c = \max_o p(o)$. Questions with $c < \tau$ are asked again, with the same state and options, to a stronger answerer: a frontier LLM forced to pick an option, or a person. The answer replaces the original, and nothing else in the procedure changes. In a cold start this lets annotators answer only the decisions the procedure is unsure of, instead of drawing graphs [E5].

4.3 Reflection

A round on a training document runs four steps with a strong LLM as the *reflector*. **Find**: for each sentence, given the gold and predicted graphs, the variable alignment, both document levels, the judge's failed units and the execution trace (with every question and its distribution), list each difference, why it is wrong, the responsible node, and the level of the cause (a wrong answer, a missing candidate or question, a wrong assembly rule, an upstream error, or doubtful gold). **Diagnose**: for each *place* (a decision skill, the shared library, or the structure), read its implementation and all its problems, and propose ranked changes with mechanism, evidence, expected effect and risk. **Edit**: turn each proposal into anchored edits (*modify*, *add* or *delete* at a unique anchor, *write* a file, or *tune* a constant over listed values). **Accept**: apply the edits to a copy of Π, which becomes the next, frozen version. Acceptance takes two forms.

Unmeasured (fragment mode). A frontier-LLM parse takes minutes and dollars, so an edit is accepted if its anchor matches exactly once and the skill still loads. Edits target instructions and examples. Rule nodes, registry and schemas are left alone, and the reflector is told to fix patterns, not instances. The next document, parsed before reflecting on it, tests the round.

Measured (closed mode). A decision model parses a document in seconds, allowing paired before/after measurements of proposed edits on a fixed set of documents. These adaptive selection trials are not independent tests of generalization. Prioritized proposals are applied one at a time on top of those already accepted, and the training, regression and development documents are re-parsed after each. An edit is kept if J rises by at least 0.1 (or C by 0.15 for a document-level-only change), mean F and mean C each fall by at most 0.3, no document's F falls by more than

$\max(0.8, 2\Delta J)$ (0.8 is about the run-to-run noise; the drop is capped at 2.5), and the regression documents fall by at most 0.3 on average. A rejected edit is sent back once with its measurement for a corrected attempt. Here the edit space is wide. Reflection may reword or add questions, add or prune options, split a long option list into a coarse and a fine question, change assembly rules, tune thresholds, add local resources, or rewire the registry. Measurement filters proposals on the selection documents, but does not guarantee stable performance on unseen documents.

Guards. The reflector never sees test documents, and test scores come only from frozen versions. The available closed-mode implementation filters six-character Chinese substrings from a configured corpus subset; complete coverage of all test documents by later run-specific versions remains to be audited [E13]. Any test-source rejection filter is an indirect use of test text and must be disclosed separately from the exclusion of test gold and scores from reflection. This check cannot rule out paraphrase or instance-specific rules. In fragment mode this was a convention rather than an automatic check [TODO: run the check over the fragment-mode versions].

5 Experimental Setup

Data. We use the 31 Chinese UMR gold documents available to us (712 sentences; 8–82 per document), preprocessed as in Chun and Xue (2024). The stored AnCast scores in the accompanying rescoring snapshot use the same gold revision; fine-grained judge analyses still require a separate revision audit and recomputation.[1] Experiments were run in stages, so we report three test sets: T1 = 0004–0005 (99 sentences) and T2 = 0005–0007 (157) for the fragment-mode runs, and H = 0015–0018, 0021–0024 (138) for the closed-mode runs. H is reported as excluded from reflection and hand tuning in these closed-mode runs. The authors have inspected H across frozen versions for analysis; it must not now be used to choose a final version or tune future edits without a new untouched test set. Training documents come from 0001–0014, and the development documents are 0027–0029 (47 sentences). Moving every system onto H is pending [E1].

[1] Some runs were first scored against an earlier revision. Re-scoring every stored prediction moved no macro average by more than 0.3 F.

Procedures. The fragment seed R_0 was written by hand from the UMR guidelines. R_{xy} is the version after round y on training document x. The closed seed J_0 uses the same registry. Its rules and options were tuned by hand on 0001–0003, so we test it only on documents from 0004 on. J_1 and J_2 follow unmeasured reflection on 0001 and 0002. K_1–K_4 continue from $K_0 = J_2$ with measured reflection on 0008–0011, one document per round.

Gold exposure. The table's reflection counts exclude manual seed tuning. J_0, J_2 and K_0 inherit exposure to 0001–0003 (three unique documents, 142 sentences). K_2 adds 0008–0009 (five unique documents, 162 sentences), and K_4 also adds 0010–0011 (seven, 199). Round 4 additionally uses three development documents (47 sentences) for adaptive selection. These are different uses of gold and must be reported separately; the closed runs are not a two-document-total-data experiment.

Models. Fragment decisions are made by Claude Sonnet 5.5, by DeepSeek V4.1 Flash (test runs without thinking), and by Qwen3.5-9B and Qwen3.8-27B (Yang et al., 2025) served with vLLM. Closed decisions are answered by three decision-model services that share one protocol: Jev (TypeSafe, 2026), the OpenAI Decisions API with `gpt-6-luna` (OpenAI, 2026), and Microsoft-Decision-1, a post-trained Qwen3.5-9B (Microsoft, 2026). The reflector is Claude Opus 5.5 unless the parsing model reflects on itself.

Baselines. *Single call*: one call per sentence with the sentence, a request for its UMR graph and a pointer to the guidelines (no document level). *Detailed single call*: a full description of the target format, plus a document-level call. Pending [E2]: a single call with the full guideline and gold examples from the same training documents; a generic tool-using agent; and the pipeline of Chun and Xue (2024).

Metrics. AnCast++ (Sun and Xue, 2024): sentence-graph F-score (F); modal, temporal and coreference scores; and the comprehensive score (Comp.). Each configuration was run once. A cold re-run changes F by up to 0.6 in closed mode and by 2.3 on average for Qwen3.5-9B in fragment mode. These limited reruns do not establish a universal noise threshold or the significance of any reported difference. Repeated runs are pending [E3].

6 Results

6.1 Orchestration versus generating the annotation

Table 1 gives the main results. Asked for a whole graph, Sonnet 5.5 reaches 29–33 F and writes no document level. The hand-written procedure, with the same model making only local decisions, starts at 45.3 F on T1 and adds the document level (Comp. 44.0 vs. 27.7). For a 9B model, orchestration makes the difference between nothing and something. Single calls yield no scorable graph on any of 24 documents, and a detailed single call reaches 4.9 F on H. Inside the procedure the same model reaches 17.6 F, ahead of the detailed call on all 21 documents where both were run. Deterministic assembly reduces structural errors but does not guarantee correct or universally parseable graphs: the format-check result must be recomputed against the current gold [E13]. Whether a generic agent with the same guideline and tools closes the gap, and at what cost, is pending [E2].

6.2 Who answers the decisions

Because the decision interface is fixed, very different models can serve the same procedure (Table 2). In fragment mode, Qwen3.8-27B reaches 42.4 F on 0001–0004, 94% of Sonnet's 45.1. DeepSeek V4.1 Flash depends on its reasoning: on 0002 it scores 49.1 F with thinking and 35.9 without (Sonnet: 50.6), so its no-thinking test runs in Table 1 understate it. In closed mode the model never writes structure, and a 9B decision service reaches 30.8 F on H, compared with 17.6 for the fragment-mode 9B configuration. This comparison changes both the answerer and the procedure version, so it does not isolate the effect of the closed interface. A procedure tuned for one answerer does not transfer unchanged, though: K_0, built with Jev (39.5), loses 8.7–13.9 F with the other two decision models. Part of the gap is calibration (§6.4).

6.3 Learning from a handful of documents

Fragment mode. With Sonnet parsing and Opus reflecting, one training document (51 sentences) raises T1 from 45.3 to 50.8 F, and a second raises it to 52.6 (Table 1). Temporal relations improve most (29.5 → 43.6). A third and fourth document leave test scores flat. The first parse of each new document, with a version that never saw it, tells the same story: 0002 goes from 50.6 (seed) to 58.9 (R_{13}), and 0003 from 49.7 to 57.5 (R_{23}; trajectories in Appendix B). Weaker models can reflect on themselves. DeepSeek gains 5.2 F on T2 from two documents, and Qwen3.8-27B gains 11.1 F on its training document in one round. The reported Qwen3.5-9B joint parser–reflector configuration fails to improve (§7).

Closed mode. Two rounds of unmeasured reflection raise the Jev procedure on T1 (36.8 → 40.2) and T2 (34.5 → 38.9). Document 0003 was used for manual seed tuning and is not an untouched test document; any wider sequential summary must exclude or disclose that exposure. Measured reflection then continues on new documents (Table 3), accepting 6–8 of 12 proposals per round. On H, the verified snapshot gives 39.5 F for K_0, 40.9 for K_3 and 40.1 for the last reported round K_4; intermediate K_1 and K_2 values await source verification [E13], while the training documents gain an order of magnitude more (+4.1 to +16.8; see §7). Learning curves over several document orders are pending [E3].

6.4 Threshold adaptation and routing

Thresholds do not transfer; recalibration is cheap. K_0's thresholds were tuned with Jev. Answered by the OpenAI Decisions API, it scores 25.6 F on H. Coordinate descent on four gold documents (0001–0003, 0006) finds improvements for two of the ten thresholds in the tested grid: when a span counts as a single-verb clause (0.6 → 0.7) and the floors for attaching an out-of-clause argument (0.4/0.6 → 0.3/0.5). The calibration objective rises from 29.0 to 30.8, and on H, which calibration never saw, F rises from 25.6 to 28.5 (Comp. 27.0 → 29.6). No value of the other eight improves the objective by more than 0.05 (Appendix C). This grid search found improvements at two thresholds. It is task-level threshold adaptation, not calibration of answer probabilities; the search does not prove that the other thresholds are optimal or causally irrelevant. Calibrating Microsoft-Decision-1, and calibrating from one or two documents, are pending [E4].

Confidence predicts agreement. On J_2's questions for two unseen documents (0010, 0012), Jev and Sonnet agree on 95% of the questions where Jev's confidence is $\geq$0.9 (53% of questions), 79% at 0.7–0.9 (26%), and 57% below 0.7 (21%). Disagreement concentrates in `sent_type`'s routing question (78%) and `np_phrase`'s modifier questions (67%), where a wrong answer reshapes a

System	Decisions by	Refl. docs (sent.)	T1: 0004–05 F/ Comp.	T2: 0005–07 F/ Comp.	H: 0015–24 F/ Comp.
Baselines: one call per sentence					
Single call	Sonnet 5.5	0	32.7 / 27.7	29.4 / 24.9	–
Single call	Qwen3.5-9B	0	–	–	0.0 / 0.0
Detailed single call	Qwen3.5-9B	0	–	–	4.9 / 5.3
Full guideline + gold examples [E2]	Sonnet 5.5	4 (201)	–	–	–
Generic tool-using agent [E2]	Sonnet 5.5	0	–	–	–
Fragment-mode orchestration (unmeasured reflection by Opus 5.5)					
R_0 (seed)	Sonnet 5.5	0	45.3 / 44.0	–	–
R_{13}	Sonnet 5.5	1 (51)	50.8 / 49.4	–	–
R_{23}	Sonnet 5.5	2 (111)	**52.6 / 51.2**	–	–
R_{32}	Sonnet 5.5	3 (142)	52.6 / 51.1	55.2 / 52.1	–
R_{42}	Sonnet 5.5	4 (201)	n/a	**55.6 / 52.2**	–
R_0 (seed)	DeepSeek V4.1‡	0	–	30.8 / 29.0	–
R_{42} (self-reflection)	DeepSeek V4.1‡	2 (110)	–	36.0 / 34.0	–
R_0 (seed)	Qwen3.5-9B	0	–	–	17.6 / 18.5
Closed-mode orchestration					
J_0 (seed)	Jev	0†	36.8 / 37.1	34.5 / 33.8	–
$J_2 = K_0$ (unmeasured reflection)	Jev	2 (111)	40.2 / 41.3	38.9 / 38.0	39.5 / 38.4
K_2 (intermediate, source audit [E13])	Jev	4 (131)	–	–	– / –
K_4 (last reported round)	Jev	6 (168)	–	–	40.1 / 39.2

Table 1: Main results on Chinese UMR (AnCast++, macro average over documents; F = sentence-graph F, Comp. = comprehensive score including the document level). "Refl. docs" counts automatic-reflection documents only, not all gold exposure. Closed seeds and their descendants additionally inherit manual tuning on 0001–0003 (see §5); round 4 also selects using development scores. "n/a": 0004 was a training document. †J_0's rules were tuned by hand on 0001–0003. ‡Parsed without thinking (§6.2). Red dashes are pending experiments ([E1], [E2]); "–" marks a cell not planned.

Answerer	Mode	F	Comp.
Qwen3.5-9B	fragment, R_0	17.6	18.5
Qwen3.8-27B	fragment, R_0	–	–
Sonnet 5.5	fragment, R_0	–	–
Microsoft-Decision-1 (9B)	closed, K_0	30.8	31.6
OpenAI Decisions	closed, K_0	25.6	27.0
+ calibrated (§6.4)	closed, K_0^θ	28.5	29.6
Jev	closed, K_0	39.5	38.4
Sonnet 5.5	closed, K_0	–	–

Table 2: One procedure, different answerers, on H. Pending: [E1], [E5].

Round (doc, sent.)	acc./tried	train F	reg. ΔF	H F
K_0				39.5
K_1 (0008, 10)	8/12	40.0→54.1	+0.5	–
K_2 (0009, 10)	7/12	43.4→60.2	+0.5	–
K_3 (0010, 12)	8/12	41.4→45.9	+0.4	40.9
K_4 (0011, 25)*	6/12	37.5→41.6	+3.2	40.1

Table 3: Reported historical trial summaries for measured reflection with Jev; source-log verification is pending [E13]. "reg. ΔF": mean change on the regression documents. *From round 4 on, problems are diagnosed on the training and regression documents together, and three development documents are measured in every trial (mean +0.8 F on them).

subtree. Agreement with Sonnet is not accuracy against gold or calibrated confidence. Reliability against gold-derived answers is pending [E4].

Routing. Routing every question with confidence below $\tau = 0.7$ from Jev to Sonnet (Table 4) recovers 3.3 of the 4.4 F that Sonnet gains by answering everything (75%), with 39% of its calls and 26% of its cost. The curve over τ, and a cold-start variant in which routed questions are answered from gold as a stand-in for an annotator, are pending [E5].

6.5 A second benchmark

[TODO: E9, to be run.] We will test whether the recipe transfers beyond UMR on verbal multiword expression identification in PARSEME 1.2 (Ramisch et al., 2020), whose guidelines are almost literally decision trees of linguistic tests, and with official evaluation of expressions unseen in training (Appendix F).

7 Analysis

Traces localize discrepancies. The trace records the producer of each predicted unit, while gold-only

Answerer	F	Sonnet calls	Cost
Jev only	39.1	0	<$0.1
Jev; $c < 0.7$ to Sonnet	42.4	504	≈$9
Sonnet only	43.5	1,292	≈$34
other τ, on H [E5]	–	–	–
$c < \tau$ from gold [E5]	–	–	–

Table 4: Routing uncertain closed decisions of J_2 on 0010+0012 (20 sentences). Questions and rules are the same in every row; only the answerer changes. Public API prices.

missing units remain unassigned. This supports inspection of candidate omissions and error propagation, but a producer is not necessarily the causal source or best repair target. Fine-grained judge counts and scores in Table 6 require recomputation against the current gold revision; their cells are left blank rather than interpreted as current evidence [E13].

Replacing an answerer under fixed questions and rules. On 0010 and 0012 with J_2's questions and rules, replacing Jev answers with Sonnet raises F from 39.1 to 43.5. This paired pilot measures the answerer's effect under those questions and rules. It cannot locate the remaining bottleneck by comparison with the fragment seed's 45.3 on T1, a different set of documents. Candidate coverage, rules and tree shape remain possible constraints, to be tested with same-document interventions and oracle answers [E6]. Among the 29 accepted measured edits, 22 change assembly rules, four option sets, one adds a tool, one a question and one changes wording; these counts describe what the optimizer changed, not the causal share of each component in the gains.

Measurement filters proposed edits. Unmeasured reflection is not filtered. In fragment mode each round finds 376–902 problems (29–43% at the document level) and applies 35–73 edits, and only 1 of 441 edits is rejected. The six skills grow from 41k to 156k characters over ten rounds, the same accumulation of rules that makes hand-maintained guidelines hard to use. The following closed-mode trial diagnostics are historical log summaries whose original run artifacts and scorer versions remain to be checked [E13]. With measurement, 19 of 48 proposals in four rounds were reverted, among them third-round attempts to *undo* earlier fixes, which the regression documents rejected. A retry with feedback often rescues an edit; one edit that first dropped a document by 2.95 F was rewritten into one that raised the objective by 1.43 with no document falling.

Training gains exceed observed test gains. The measured-reflection trajectory has larger improvements on documents used for diagnosis and selection than on H. Round 4 measured three development documents and improved their mean by 0.8 F, yet the H mean declined from 40.9 to 40.1. Limited cold reruns are insufficient to rule out stochastic variation. Document-specific conventions, restricted development coverage, adaptive selection and answerer behavior are possible explanations; the current trajectory does not separate them. We show the last reported round and intermediate trajectory; this does not establish a pre-registered checkpoint-selection policy. Choosing a version using H would require a new untouched final test set. Repeated trajectories and matched acceptance-policy ablations remain pending [E3], [E6], [E8].

Weak reflectors and cost. With Qwen3.5-9B as parser and reflector, each new document parsed with the previous version is 1.4 F *below* the seed on average (noise: 2.3). The 9B reflector rewrites the anchors it should copy (1 of 15 edits applied) and writes noisy rules, so attribution alone is not enough. As for cost, fragment mode with Sonnet takes dozens of calls per sentence ($0.71 per sentence on T1), whereas closed mode took 7,552 requests and about $0.64 for 216 sentences, and four rounds of measured reflection about $8 in decision requests plus 36–82 reflector calls per round [E11].

8 Conclusion

Decision orchestration separates deterministic dispatch and assembly from local model judgments. In the reported Chinese UMR runs, supervised artifact updates, task-level threshold tuning and decision routing improve several configurations without weight updates. Traces support inspection, not causal attribution. Training gains exceed test gains; matched baselines, repeated trajectories and targeted ablations remain necessary to establish the robustness and contribution of each component.

Limitations

Our main evidence comes from one task in one language. The second benchmark (§6.5) has not

been run yet, and the English UMR runs, made with earlier versions of the procedure, are not reported. Test sets are small (99–157 sentences) and differ between stages of the work. Unifying them is planned ([E1]). Most configurations were run once, and run-to-run noise (up to 0.6 F for closed mode and 2.3 F for a 9B model in fragment mode) is of the same order as some of the differences we report. The seeds were written by the authors, so their quality, and the effort it took to write them, affect every result. We have not measured that effort. Several of the models we use, including all three decision-model services and the frontier LLMs, are commercial APIs whose behavior may change; the release must include shareable predictions, exact versions and permitted cached answers before these runs can be called independently reproducible [TODO: check that the release includes them]. The routing results come from 20 sentences, and the "human" in routing will be simulated from gold, which overstates real annotators' accuracy. Finally, AnCast and our fine-grained judge depend on an automatic alignment, and the gold itself contains errors. One gold graph has unbalanced brackets and is skipped by the scorer.

Ethics Statement

We use the Chinese UMR annotations supplied in the research checkout. [TODO: Verify the exact release, license, cleaned-gold provenance, redistribution and commercial-API use permissions.] We collected no new data from people and no personal information beyond what the released documents contain. Our goal is to reduce the cost of producing the first annotations for a new scheme, which could reduce demand for annotation work. We see the method as a way to direct annotators' effort to the decisions that need human judgment. Running the experiments in this paper used commercial APIs, with a complete cost and price-date audit still pending [TODO: sum the API costs].

Generative AI assisted author-side simulated review and manuscript editing. It did not supply new experimental results; this assistance is not formal ARR peer review.

References

Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *The Fourteenth International Conference on Learning Representations*.

Xuefeng Bai, Yulong Chen, and Yue Zhang. 2022. Graph pre-training for AMR parsing and generation. *arXiv preprint arXiv:2203.07836*.

Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2023. Prompting is programming: A query language for large language models. *Proceedings of the ACM on Programming Languages*, 7(PLDI):1946–1969.

Michele Bevilacqua, Rexhina Blloshmi, and Roberto Navigli. 2021. One SPRING to rule them both: Symmetric AMR semantic parsing and generation without a complex pipeline. *Proceedings of the AAAI Conference on Artificial Intelligence*, 35(14):12564–12573.

Julia Bonn, Claire Bonial, Matt Buchholz, Hsiao-Jung Cheng, Alvin Chen, Ching-wen Chen, Andrew Cowell, William Croft, Lukas Denk, Ahmed Elsayed, Eva Fučíková, Federica Gamba, Carlos Gomez, Jan Hajič, Eva Hajičová, Jiří Havelka, Loden Havenmeier, Ath Kilgore, Veronika Kolářová, and 40 others. 2025. Uniform meaning representation 2.0.

Julia Bonn, Matthew J. Buchholz, Jayeol Chun, Andrew Cowell, William Croft, Lukas Denk, Sijia Ge, Jan Hajič, Kenneth Lai, James H. Martin, Skatje Myers, Alexis Palmer, Martha Palmer, Claire Benet Post, James Pustejovsky, Kristine Stenzel, Haibo Sun, Zdeňka Urešová, Rosa Vallejos, and 4 others. 2024. Building a broad infrastructure for uniform meaning representations. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 2537–2547, Torino, Italia. ELRA and ICCL.

Julia Bonn, Chen Ching-wen, James Andrew Cowell, William Croft, Lukas Denk, Jan Hajič, Kenneth Lai, Martha Palmer, Alexis Palmer, James Pustejovsky, Haibo Sun, Rosa Vallejos Yopán, Jens Van Gysel, Meagan Vigus, Nianwen Xue, and Jin Zhao. 2023. Uniform meaning representation. LINDAT/CLARIAH-CZ digital library at the Institute of Formal and Applied Linguistics.

Eric Brill. 1995. Transformation-based error-driven learning and natural language processing: A case study in part-of-speech tagging. *Computational Linguistics*, 21(4):543–565.

Lingjiao Chen, Matei Zaharia, and James Zou. 2024. FrugalGPT: How to use large language models while reducing cost and improving performance. *Transactions on Machine Learning Research*.

Jayeol Chun and Nianwen Xue. 2024. A pipeline approach for parsing documents into uniform meaning representation graphs. In *Proceedings of the Fifth International Workshop on Designing Meaning Representations @ LREC-COLING 2024*.

Allyson Ettinger, Jena D. Hwang, Valentina Pyatkin, Chandra Bhagavatula, and Yejin Choi. 2023. "you are an expert linguistic annotator": Limits of llms as analyzers of abstract meaning representation. *arXiv preprint arXiv:2310.17793*.

Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 10764–10799. PMLR.

Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. 2023. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 10932–10952, Singapore. Association for Computational Linguistics.

Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. 2023. ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30):e2305016120.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330. PMLR.

Tanmay Gupta and Aniruddha Kembhavi. 2023. Visual programming: Compositional visual reasoning without training. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14953–14962.

Xingwei He, Zhenghao Lin, Yeyun Gong, A-Long Jin, Hang Zhang, Chen Lin, Jian Jiao, Siu Ming Yiu, Nan Duan, and Weizhu Chen. 2024. AnnoLLM: Making large language models to be better crowdsourced annotators. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, pages 165–190, Mexico City, Mexico. Association for Computational Linguistics.

Shengran Hu, Cong Lu, and Jeff Clune. 2025. Automated design of agentic systems. In *The Thirteenth International Conference on Learning Representations*.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations*.

Zhengbao Jiang, Jun Araki, Haibo Ding, and Graham Neubig. 2021. How can we know when language models know? on the calibration of language models for question answering. *Transactions of the Association for Computational Linguistics*, 9:962–977.

Amita Kamath, Robin Jia, and Percy Liang. 2020. Selective question answering under domain shift. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 5684–5696, Online. Association for Computational Linguistics.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, and 1 others. 2024. DSPy: Compiling declarative language model calls into self-improving pipelines. In *International Conference on Learning Representations*.

Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. 2023. Decomposed prompting: A modular approach for solving complex tasks. In *The Eleventh International Conference on Learning Representations*.

Kevin Knight, Bianca Badarau, Laura Baranescu, Claire Bonial, Madalina Bardocz, Kira Griffitt, Ulf Hermjakob, Daniel Marcu, Martha Palmer, Tim O'Gorman, and Nathan Schneider. 2020. Abstract Meaning Representation (AMR) Annotation Release 3.0. LDC2020T02.

Minzhi Li, Taiwei Shi, Caleb Ziems, Min-Yen Kan, Nancy Chen, Zhengyuan Liu, and Diyi Yang. 2023. CoAnnotating: Uncertainty-guided work allocation between human and large language models for data annotation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 1487–1505, Singapore. Association for Computational Linguistics.

Emma Markle, Javier Gutierrez Bach, and Shira Wein. 2025. Setup: Sentence-level english-to-uniform meaning representation parser. *arXiv preprint arXiv:2512.07068*.

Microsoft. 2026. Use Microsoft-Decision-1 in Azure AI Foundry. `https://learn.microsoft.com/en-us/azure/foundry/foundry-models/how-to/use-foundry-models-microsoft-decision`. Accessed October 2026.

OpenAI. 2026. Decisions API guide. `https://developers.openai.com/api/docs/guides/decisions`. Accessed October 2026.

Krista Opsahl-Ong, Michael J. Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. 2024. Optimizing instructions and demonstrations for multi-stage language model programs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9340–9366, Miami, Florida, USA. Association for Computational Linguistics.

Martha Palmer, Daniel Gildea, and Paul Kingsbury. 2005. The proposition bank: An annotated corpus of semantic roles. *Computational Linguistics*, 31(1):71–106.

Chaoxu Pang, Yixuan Cao, Qiang Ding, and Ping Luo. 2023. Guideline learning for in-context information extraction. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 15372–15389, Singapore. Association for Computational Linguistics.

Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. 2023. Automatic prompt optimization with "gradient descent" and beam search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*.

Carlos Ramisch, Agata Savary, Bruno Guillaume, Jakub Waszczuk, Marie Candito, Ashwini Vaidya, Verginica Barbu Mititelu, Archna Bhatia, Uxoa Iñurrieta, Voula Giouli, Tunga Güngör, Menghan Jiang, Timm Lichte, Chaya Liebeskind, Johanna Monti, Renata Ramisch, Sara Stymne, Abigail Walsh, and Hongzhi Xu. 2020. Edition 1.2 of the PARSEME shared task on semi-supervised identification of verbal multiword expressions. In *Proceedings of the Joint Workshop on Multiword Expressions and Electronic Lexicons*, pages 107–118. Association for Computational Linguistics.

Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. 2017. Snorkel: Rapid training data creation with weak supervision. *Proceedings of the VLDB Endowment*, 11(3):269–282.

Oscar Sainz, Iker García-Ferrero, Rodrigo Agerri, Oier Lopez de Lacalle, German Rigau, and Eneko Agirre. 2024. GoLLIE: Annotation guidelines improve zero-shot information-extraction. In *The Twelfth International Conference on Learning Representations*.

Manon Scholivet, Agata Savary, Carlos Ramisch, Eric Bilinski, Takuya Nakamura, Maria Mitrofan, and Vasile Pais. 2026. Edition 2.0 of the PARSEME shared task on multilingual identification and paraphrasing of multiword expressions. In *Proceedings of the 22nd Workshop on Multiword Expressions (MWE 2026)*, pages 254–275. Association for Computational Linguistics.

Burr Settles. 2009. Active learning literature survey. Computer Sciences Technical Report 1648, University of Wisconsin–Madison.

Haibo Sun and Nianwen Xue. 2024. Anchor and broadcast: An efficient concept alignment approach for evaluation of semantic graphs. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 1052–1062, Torino, Italia. ELRA and ICCL.

Haibo Sun, Nianwen Xue, Jin Zhao, Liulu Yue, Yao Sun, Keer Xu, and Jiawei Wu. 2024. Chinese UMR annotation: Can LLMs help? In *Proceedings of the Fifth International Workshop on Designing Meaning Representations @ LREC-COLING 2024*, pages 131–139.

Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. 2024. Let me speak freely? a study on the impact of format restrictions on large language model performance. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 1218–1236, Miami, Florida, US. Association for Computational Linguistics.

Zhen Tan, Dawei Li, Song Wang, Alimohammad Beigi, Bohan Jiang, Amrita Bhattacharjee, Mansooreh Karami, Jundong Li, Lu Cheng, and Huan Liu. 2024. Large language models for data annotation and synthesis: A survey. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 930–957, Miami, Florida, USA. Association for Computational Linguistics.

TypeSafe. 2026. Jev API reference. `https://docs.typesafe.ai/api`. Accessed October 2026.

Jens E. L. Van Gysel, Meagan Vigus, Jayeol Chun, Kenneth Lai, Sarah Moeller, Jiarui Yao, Tim O'Gorman, Andrew Cowell, William Croft, Chu-Ren Huang, Jan Hajič, James H. Martin, Stephan Oepen, Martha Palmer, James Pustejovsky, Rosa Vallejos, and Nianwen Xue. 2021. Designing a uniform meaning representation for natural language processing. *KI - Künstliche Intelligenz*, 35(0):343–360.

Shuohang Wang, Yang Liu, Yichong Xu, Chenguang Zhu, and Michael Zeng. 2021. Want to reduce labeling cost? GPT-3 can help. In *Findings of the Association for Computational Linguistics: EMNLP 2021*, pages 4195–4205, Punta Cana, Dominican Republic. Association for Computational Linguistics.

Xingyao Wang, Sha Li, and Heng Ji. 2023. Code4Struct: Code generation for few-shot event structure prediction. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3640–3663, Toronto, Canada. Association for Computational Linguistics.

Xinru Wang, Hannah Kim, Sajjadur Rahman, Kushan Mitra, and Zhengjie Miao. 2024. Human-LLM collaborative annotation through effective verification of LLM labels. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)*, pages 1–21, Honolulu, HI, USA. ACM.

Brandon T. Willard and Rémi Louf. 2023. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*.

Tongshuang Wu, Michael Terry, and Carrie Jun Cai. 2022. AI chains: Transparent and controllable human-AI interaction by chaining large language model prompts. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems (CHI '22)*, pages 1–22, New Orleans, LA, USA. ACM.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao,

Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. In *International Conference on Learning Representations*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. Optimizing generative AI by backpropagating language model feedback. *Nature*.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. 2024. Self-taught optimizer (STOP): Recursively self-improving code generation. In *First Conference on Language Modeling*.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2025. AFlow: Automating agentic workflow generation. In *The Thirteenth International Conference on Learning Representations*.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations*.

Zihao Zhao, Eric Wallace, Shi Feng, Dan Klein, and Sameer Singh. 2021. Calibrate before use: Improving few-shot performance of language models. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 12697–12706. PMLR.

Jiawei Zhou, Tahira Naseem, Ramón Fernandez Astudillo, Young-Suk Lee, Radu Florian, and Salim Roukos. 2021. Structure-aware fine-tuning of sequence-to-sequence transformers for transition-based AMR parsing. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 6279–6290, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.

Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023. Large language models are human-level prompt engineers. In *International Conference on Learning Representations*.

Caleb Ziems, William Held, Omar Shaikh, Jiaao Chen, Zhehao Zhang, and Diyi Yang. 2024. Can large language models transform computational social science? *Computational Linguistics*, 50(1):237–291.

A Fine-grained judge

The judge works on the AnCast alignment of predicted and gold variables. It scores each of the following checks as an F-score over *units*. A unit is one small comparison (a node, an attribute, an argument set, a relation) that is correct, partially correct, wrong, missing or extra. **(a) nodes**: event found, event concept, entity found, entity concept, granularity (is the node cut at the same words as gold); **(b)** event sense; **(c) attributes**: aspect, modal strength, polarity, refer-person/number, other attributes; **(d) arguments**: core arguments of events, core arguments of abstract predicates, peripheral arguments of events, peripheral arguments of entities; **(e) role labels**: core, peripheral; **(f) names**: name tokens, wiki; **(g)** reentrancy; **(h)** temporal; **(i)** modal; **(j)** coreference chains and subsets; **(k)** document format. For attribution, each unit is assigned to the node that produced its predicted side, or to "no predicted node" when only the gold side exists.

B Training trajectories

Doc	seed	iter 1	iter 2	iter 3	after
0001	40.3	40.3	59.6	66.9	70.9
0002	50.6	58.9	66.3	70.4	73.4
0003	49.7	57.5	69.5	–	–
0004	39.6	49.7	57.9	–	–

Table 5: Fragment mode (Sonnet 5.5 parsing, Opus 5.5 reflecting): sentence F on each training document. Iteration y parses with the version before round y, so iteration 1 uses only earlier documents. "after" parses with the document's final version.

C Task-Level Threshold Adaptation

Table 7 lists the thresholds of the closed-mode procedure and the sweep for the OpenAI Decisions API. The objective is Eq. 1 on 0001–0003 and 0006. Thresholds were swept in the order listed, each on top of the values kept so far, and a value is kept if it improves the objective by more than 0.05.

D Skills

A fragment-mode skill. `predicate/skill.md` has sections on the head concept (event, abstract

Skill	iter 1	iter 2	iter 3	after
`predicate`	–	–	–	–
`np_phrase`	–	–	–	–
`document`	–	–	–	–
(no predicted node)	–	–	–	–
`sent_type`	–	–	–	–
`special_entity`	–	–	–	–
`discourse`	–	–	–	–

Table 6: Credit lost per skill on training document 0001 over three rounds of fragment-mode reflection (sum of $1 - \text{credit}$ over all judge units attributed to the skill). Cells are intentionally blank pending current-gold recomputation and scorer-version audit [E13]; this table is a planned diagnostic, not a result.

Threshold (skill)	Jev	tried	kept (ΔJ)
comma span → discourse (`sent_type`)	.2	.3 .4 .5	.2
split noun phrase (`sent_type`)	.2	.3 .4 .5	.2
word is an event (`sent_type`)	.5	.4 .6 .7	.5
single-verb clause (`sent_type`)	.6	.4 .5 .7	**.7** (+1.55)
conjunct cut, and/or (`discourse`)	.4/.5	+.1, +.2, +.3	.4/.5
cut without comma (`discourse`)	.5	.6 .7 .8	.5
outside argument, ARG0-1/other (`predicate`)	.4/.6	±.1, +.2	**.3/.5** (+0.27)
generic yes/no (shared)	.5	.6 .7 .8	.5
coreference kept (`document`)	.4	.3 .5 .6	.4
modal source kept (`document`)	.7	.5 .6 .8	.7

Table 7: Calibration of K_0 for the OpenAI Decisions API. Objective 29.0 → 30.8 on the calibration documents; on H, F 25.6 → 28.5. [TODO: same table for Microsoft-Decision-1 (E4)]

predicate, sense), the participants, participants outside the clause, aspect, modal strength and polarity, the output, worked examples, and, last, an `## Input` section with the span, the numbered sentence and the outputs of the parent node. Because the input comes last, everything before it is cached across calls. Reflection edits these sections in place by anchored edits. The emit schema lists closed value sets for aspect (27 values), modal strength (6) and polarity, and open fields for the fragment, its open slots and its out-of-span participants. Over ten rounds of unmeasured reflection, the six decision skills grow from 41k to 156k characters of instructions.

A closed-mode skill. A closed skill is a Python generator:

```
def run(sent, span, **_):
    a = yield {"state": {...}, "questions": {
        "type": {"type": "choice",
                 "criteria": TYPES}}}
    b = yield {...}   # may depend on a
    return emit_from(a, b)  # rules
```

Each option in `TYPES` carries a description condensed from the guideline, for example what counts as a *discourse* span in Chinese, with the constructions that do and do not qualify. The returned emit has the same schema as the fragment-mode skill, so

Node	Kind	Decides or does
`sent_type`	decision	span type (5-way); routing
`discourse`	decision	relation; split points
`predicate`	decision	head, sense, roles, aspect, modality
`np_phrase`	decision	head, its form, modifier roles
`special_entity`	decision	entity kind, type, value
`document`	decision	time anchor, source, coreference
`antecedent`	rule	out-of-span participants
`coverage`	rule	re-dispatch uncovered tokens
`reentrancy`	rule	merge same-referent nodes
output handler	rule	assemble, validate

Table 8: Nodes of the UMR procedure. Each decision node has two interchangeable implementations (fragment and closed); rule nodes, registry and scorer are shared.

the registry and rule nodes are unchanged. Table 9 lists the questions per skill.

Skill	Rounds of closed questions
`sent_type`	span type (5-way); for a one-word span in PropBank, its use here (event, state, manner, thing)
`discourse`	relation (8 frequent ones); one yes/no per possible cut point
`predicate`	(1) head word, verb or abstract predicate; (2) sense, aspect, modal strength, negation; (3) role of each content word; antecedent of each core argument outside the clause
`np_phrase`	(1) head and its form (8-way); (2) role of each modifier; sense if the head is an event
`special_entity`	kind (name, date, quantity, percentage, ordinal); entity type (44); quantity type
`document`	(1) temporal relation to the anchor event; source of a view; earlier mention it co-refers with, or new; (2) confirm each coreference (yes/no)

Table 9: Closed questions asked by each decision skill of J_0. Values (numbers, dates, names) are read by rules.

E Reflection prompts

[TODO: condense rsi/find_problems.md, rsi/plan_edits.md and rsi-jev2/diagnose.md into half a page each; include the list of proposal kinds and the acceptance constants of §4.3]

F Second benchmark: planned procedure

Task and data. PARSEME 1.2 (Ramisch et al., 2020): identify verbal multiword expressions and their category in sentences in the `.cupt` format (CoNLL-U plus an MWE column), Chinese and one more language. We will simulate the cold start by drawing "documents" of 100 training sentences

and learning from 1, 2 and 4 of them, measuring on the official development set and testing once on the official test set. Metrics are the official MWE-based and token-based P/R/F and MWE-based F on unseen expressions.

Procedure. **Candidates (rule).** For each verb, enumerate the combinations with its dependents that a category allows: a verb with a nominal object or oblique (light-verb construction); a verb with a particle or, in Chinese, a resultative or directional complement (verb-particle construction); serial verbs (multi-verb construction, following the Chinese-specific decision tree); and a verb with lexicalized dependents (verbal idiom). Expressions seen in training are matched by lemma (rule). **Decisions (closed).** Each guideline test becomes a yes/no question about one candidate in context, for example whether the noun is predicative, whether the verb is semantically light, or whether replacing a component changes the meaning. The decision tree of the guidelines becomes code that asks the next test according to the answers. **Assembly (rule).** Spans and categories are written to `.cupt`. Overlapping candidates are resolved by rule. **Learning.** Thresholds are calibrated per test. Measured reflection may reword tests, add candidate patterns or reorder the tree.

Planned models and controls. Closed seed and adapted procedures will use Jev, with Opus for reflection. A direct Jev classifier will select a category or non-MWE for each candidate, using the same frozen candidate generator and assembler; this controls for the answerer when testing the decision tree. Sonnet guideline/few-shot and fragment configurations are separate generative-system comparisons, and optional routing escalates Jev questions to Sonnet. Quality and total construction/inference cost will be reported separately. These configurations have not been evaluated [E9].

Why PARSEME. The PARSEME guidelines are, almost literally, a decision procedure. Candidates are found on the dependency trees that ship with the data, and each candidate is classified by linguistic tests organized as decision trees into a closed set of categories (verbal idioms, light-verb constructions, inherently reflexive verbs, verb-particle and multi-verb constructions). The test sets contain about 300 expressions per language that are unseen in training, which is the regime a procedure learned from a few documents should help with. The data is freely available, has an official scorer, and includes Chinese, which keeps the language constant with our UMR experiments; we will add one language from a different family [TODO: choose: French or Polish]. Edition 2.0 extends the scheme to all syntactic categories in 17 languages (Scholivet et al., 2026), which allows a second test after a schema revision. Table 10 lists the reference numbers.

PARSEME 1.2, Chinese test	unseen F1	global F1
Two-rule baseline (Ramisch et al., 2020)	27.3	–
Best 1.2 system on ZH (TRAVIS-mono)	60.2	–
Single LLM call with guidelines [E9]	–	–
Orchestration, seed [E9]	–	–
+ 1/2/4 training documents [E9]	–	–

Table 10: Planned second benchmark. Reference numbers are from Ramisch et al. (2020). Global F1 for the reference systems will be taken from the official result tables.

Baselines. The two-rule baseline and the official results of edition 1.2; a single LLM call with the guidelines (zero-shot and with the same training sentences as examples); and the fragment-mode variant of the same procedure.

Planned generalization check. To distinguish useful supervised reflection from fitting the diagnosis documents, we will compare the seed, the current reflection protocol and a cross-document selection protocol under matched gold, reflector and proposal budgets. The latter will propose edits from recurring training errors, state their guideline basis and applicability, and evaluate them on training-pool documents excluded from that proposal's diagnosis. Regression effects, instruction growth and cost will also be recorded. These selection documents are not final-test data. Checkpoint selection will use development data under a fixed query budget; frozen versions will be evaluated with repeated runs on a separate final set. The previously inspected H trajectory remains retrospective evidence, rather than a fresh confirmation set [E3], [E6], [E8].

G Pending experiments (to be removed before submission)

E1 One gold revision, one held-out set (H), one development set for every system in Table 1.

E2 Stronger baselines: full guideline with gold examples from the same training documents; a

generic tool-using agent; Chun and Xue (2024).

E3 Learning curves over 0/1/2/4/8 documents with three document orders.

E4 Calibration: reliability diagrams and ECE against gold-derived answers for three decision models; calibrating Microsoft-Decision-1; calibration from 1, 2 and 4 documents; transfer of thresholds between models.

E5 Routing: the full curve over τ on H; routed questions answered from gold as a simulated annotator; Sonnet as the only closed answerer on H.

E6 Measured reflection: ablation of round 4, a larger and more varied development set, rounds 5–7, and the same loop with the other two decision models.

E7 Ablations of the procedure: without the coverage and reentrancy rules; whole-graph generation instead of fragments; without the PropBank tool.

E8 Ablations of reflection: wording-only edits vs. edits to rules, options and structure; unmeasured vs. measured acceptance at equal budget; training-only vs. training + regression + development measurement.

E9 Second benchmark (PARSEME 1.2, Appendix F).

E10 English UMR with the same procedure.

E11 Cost table: calls, tokens, dollars and wall-clock per configuration.

E12 Annotator time: correcting the procedure's output vs. annotating from scratch.

E13 Evidence audit: recompute fine-grained judge results on the current gold; locate original K_1/K_2 predictions and scores; record scorer and model versions, gold checksums, invalid-graph handling, and the exact copying-guard coverage.

H Reproduction Status

The bundled AnCast rescoring JSON supports several reported macro averages but does not include the intermediate K_1/K_2 entries. Those H-score cells and current-gold judge counts remain blank [E13]. The rescoring notes identify predictions from a separate experimental branch and a scorer not bundled with this checkout; the main-branch scorer also has an unresolved missing-variable path for failed graph matching. Before reproduction, provide a frozen scorer, all relevant predictions, gold checksums and exclusions, code commits, service/model versions and dates, runtime settings and a complete data-exposure ledger. Do not regenerate result tables with a different gold or silently drop failed predictions. Current cached-answer availability and redistribution permissions remain to be verified; commercial API access is not needed to audit already stored predictions.